
Simulation et méthodes de Monte-Carlo

Rapport de projet : Medium Access Control Simulation d'un protocole MAC avec Exponential Backoff

Formation :
M1 Informatique

Réalisé par :
— Douadjia Abdelkarim
— BESSAA Abderraouf

Professeur :
Monsieur Pierre Coucheney

Année universitaire :
2025 – 2026

Table des matières

1	Introduction	2
2	Organisation des fichiers du projet	2
3	Modèle simulé et paramètres retenus	2
3.1	Paramètres de référence	2
3.2	Hypothèses ajoutées dans notre implémentation	3
4	Simulateur à événements discrets	3
4.1	Principe général	3
4.2	Variables d'état	4
4.3	Séries temporelles enregistrées	5
4.4	Types d'événements	5
4.5	Traitement des événements	5
4.6	Métriques finales	6
4.7	Métriques calculées	6
5	Validation du simulateur	7
5.1	Validation de la loi exponentielle	7
5.2	Validation avec un cas sans collision	7
5.3	Validation sur une charge modérée	7
6	Résultats expérimentaux	8
6.1	Evolution temporelle des métriques	8
6.2	Débit en fonction de τ	10
6.3	Influence de K sur le débit et les pertes	11
6.4	Influence de λ sur le débit	12
6.5	Influence de N sur le débit	13
7	Détermination du nombre optimal de stations	15
8	Discussion	16
9	Conclusion	16

1 Introduction

Ce rapport présente notre simulateur du protocole MAC avec *Exponential Backoff*. L'objectif est d'étudier, par simulation à événements discrets, le comportement d'un ensemble de stations partageant le même canal de communication. Les résultats présentés proviennent de l'exécution du notebook `projet_MAC.ipynb` : les courbes et les tableaux numériques correspondent directement aux sorties obtenues pendant l'exécution.

Le rapport se concentre sur les éléments utiles à l'évaluation : modèle réellement implémenté, événements du simulateur, hypothèses ajoutées, tests de validation, choix des paramètres, résultats expérimentaux et détermination du nombre de stations qui maximise le débit avec un intervalle de confiance à 95%.

2 Organisation des fichiers du projet

Le projet contient deux notebooks. Pour éviter toute confusion, nous distinguons clairement leur rôle.

TABLE 1 – Rôle des fichiers source fournis.

Fichier	Rôle
<code>projet_MAC.ipynb</code>	Notebook principal utilisé pour produire les résultats du rapport. Il contient le simulateur, les tests de validation, les courbes temporelles, l'étude selon λ , l'étude selon N , la recherche du N optimal avec IC à 95%, l'étude selon τ et l'étude selon K .
Dépôt GitHub	Code source du projet disponible à l'adresse : github.com/abdou-bs/simulation_Medium_Access_Control_2026 .
<code>simulation_mac.ipynb</code>	Notebook complémentaire utilisé pendant le développement pour tester des idées, vérifier certains comportements et comparer des versions intermédiaires du simulateur. Les conclusions numériques du rapport sont prises depuis <code>projet_MAC.ipynb</code> .
<code>figures/</code>	Dossier contenant les figures exportées depuis le notebook et intégrées dans le rapport LaTeX.

3 Modèle simulé et paramètres retenus

3.1 Paramètres de référence

Pour l'étude temporelle principale, nous avons utilisé les paramètres suivants. Ils permettent d'obtenir une charge totale modérée, égale à $N\lambda = 0.6$, afin d'observer à la fois des transmissions réussies, des collisions et une convergence du débit.

TABLE 2 – Paramètres utilisés pour la simulation de référence.

Paramètre	Valeur	Rôle dans la simulation
N	10	Nombre de stations
K	10	Capacité de la file d'attente par station
λ	0.06	Taux d'arrivée des paquets par station
τ	0.5	Paramètre de base du backoff
$T_{\max}$	3000	Durée de simulation
record_every	5	Pas d'enregistrement des métriques
seed	2026	Reproductibilité

3.2 Hypothèses ajoutées dans notre implémentation

Le modèle de l'énoncé laisse une certaine liberté d'implémentation. Nous avons donc fixé explicitement les choix suivants :

- La durée d'émission d'un paquet est constante et vaut 1 unité de temps.
- Les arrivées sont indépendantes d'une station à l'autre et les inter-arrivées suivent une loi exponentielle de paramètre λ .
- Chaque station possède une file d'attente bornée de capacité K .
- Un paquet est compté comme **perdu** uniquement lorsqu'il arrive dans une file déjà pleine. C'est cette perte qui est utilisée pour le taux de paquets perdus.
- Une collision correspond à une transmission échouée : le paquet n'est pas transmis avec succès, puis la station retente après un délai de backoff. La collision n'est donc pas comptée dans la métrique "perdus file pleine".
- Lorsqu'une station réussit une transmission, son état de backoff est réinitialisé à 1.
- Lorsqu'une station trouve le canal occupé, elle reprogramme sa tentative à l'instant où le canal redevient libre.
- Le paramètre τ contrôle l'échelle du délai de backoff : après une collision, une valeur plus grande de τ tend à espacer davantage les retransmissions.
- Les expériences comparatives sont répétées plusieurs fois avec des simulations indépendantes afin de calculer une moyenne et un intervalle de confiance à 95%.

Ces hypothèses rendent les métriques séparées et interprétables : le débit mesure les transmissions réussies, les collisions mesurent les conflits d'accès au canal, et le taux de perte mesure la saturation des files d'attente.

4 Simulateur à événements discrets

4.1 Principe général

Le simulateur est un simulateur à événements discrets. Le temps ne progresse pas avec un pas fixe ; il saute directement d'un événement au suivant. Cette approche est adaptée au projet car les changements d'état du système se produisent uniquement lors des arrivées de paquets ou lors des tentatives d'émission.

Les événements sont stockés dans un **échéancier**. Dans le code, cet échéancier est implémenté avec la structure `heapq`, qui permet d'extraire rapidement l'événement le plus proche dans le temps. Chaque élément de l'échéancier contient au minimum l'instant de l'événement, son type et la station concernée.

Initialisation de l'échéancier

```
Initialiser une file de priorite vide : events = []
Pour chaque station i :
    generer la date de sa premiere arrivee avec Exp(lambda)
    ajouter ARRIVAL(t_i, i) dans l'echeancier
Initialiser les files, les etats de backoff et les compteurs
Lancer la boucle principale jusqu'a T_max
```

Cette initialisation signifie que chaque station commence avec son propre processus d'arrivée. Ensuite, après chaque événement `ARRIVAL`, une nouvelle arrivée est planifiée pour la même station. Cela permet de conserver un flux d'arrivées pendant toute la simulation.

4.2 Variables d'état

Le simulateur conserve plusieurs variables permettant de décrire l'état courant du système.

TABLE 3 – Variables principales du simulateur.

Variable	Type	Description
<code>queue[i]</code>	entier	Nombre de paquets en attente dans la station i .
<code>backoff_state[i]</code>	entier	Etat de backoff courant de la station i . Plus il est grand, plus le délai moyen avant retransmission est élevé.
<code>pending_attempt[i]</code>	booléen	Indique si une tentative d'émission est déjà planifiée pour la station i . Cette variable évite de programmer plusieurs tentatives pour la même station.
<code>channel_busy_until</code>	réel	Instant à partir duquel le canal redevient libre. Si $t < \text{channel_busy_until}$, une station ne peut pas émettre immédiatement.
<code>total_queue_area</code>	réel	Intégrale temporelle de la somme des files d'attente. Elle sert à calculer le nombre moyen de clients dans le système.
<code>last_t</code>	réel	Dernier instant traité. Cette variable permet de calculer la durée écoulée depuis le dernier événement afin de mettre à jour <code>total_queue_area</code> .
<code>n_arrived</code>	entier	Nombre total de paquets arrivés dans le système.
<code>n_success</code>	entier	Nombre de paquets transmis avec succès.
<code>n_collision</code>	entier	Nombre de stations impliquées dans une collision.
<code>n_lost_full</code>	entier	Nombre de paquets refusés parce que la file de leur station est pleine.

Les variables `total_queue_area` et `last_t` sont importantes car la longueur moyenne de file n'est pas calculée seulement à la fin. À chaque événement, on connaît la durée pendant laquelle le

système est resté dans le même état. On ajoute donc :

$$(\text{temps courant} - \text{last_t}) \times \sum_{i=1}^N q_i$$

à `total_queue_area`. À la fin de la simulation, le nombre moyen de clients vaut :

$$\bar{L} = \frac{\text{total_queue_area}}{T_{\max}}.$$

4.3 Séries temporelles enregistrées

Pour tracer l'évolution du système dans le temps, nous enregistrons périodiquement plusieurs séries temporelles.

TABLE 4 – Séries temporelles utilisées pour les figures.

Série	Description
<code>times</code>	Instants auxquels les métriques sont enregistrées.
<code>throughputs</code>	Valeurs de $n(t)/t$, c'est-à-dire le débit cumulé à l'instant t .
<code>queue_avgs</code>	Nombre moyen de paquets en file, calculé à partir de <code>total_queue_area</code> .
<code>loss_rates</code>	Taux de perte cumulé, calculé par <code>n_lost_full</code> / <code>n_arrived</code> .

Ces séries sont enregistrées toutes les `record_every` unités de temps. Cela permet de visualiser la phase transitoire, puis le régime où les grandeurs deviennent plus stables.

4.4 Types d'événements

TABLE 5 – Événements traités par le simulateur.

Événement	Constante	Rôle
Arrivée d'un paquet	ARRIVAL	Ajout d'un paquet dans la file d'une station
Tentative d'émission	ATTEMPT	Essai d'utilisation du canal par une station

4.5 Traitement des événements

Algorithme de traitement d'une arrivée

```

A l'instant t, pour la station i :
1. Mettre à jour total_queue_area avec la durée écoulée depuis last_t.
2. Incrementer le nombre total d'arrivées.
3. Si queue[i] < K :
    ajouter le paquet dans queue[i].
    si aucune tentative n'est déjà planifiée :
        planifier ATTEMPT(t, i).
Sinon :
```

```

    incrementer n_lost_full.
4. Planifier la prochaine arrivee de la station i a t + Exp(lambda).
5. Mettre last_t = t.

```

Algorithme de traitement d'une tentative

```

A l'instant t, pour la station i :
1. Mettre a jour total_queue_area avec la duree ecoulee depuis last_t.
2. Si queue[i] == 0 : ne rien envoyer.
3. Si le canal est occupe :
    replanifier ATTEMPT(channel_busy_until, i).
4. Sinon, collecter toutes les tentatives prevues exactement a t.
5. Si une seule station tente :
    transmission reussie ; queue[i] diminue de 1 ; backoff_state[i] =
    1.
6. Si plusieurs stations tentent :
    collision ; chaque station j attend Exp(1 / (tau * 2^k)),
    puis passe de l'etat k a l'etat k+1.
7. Mettre last_t = t.

```

4.6 Métriques finales

À la fin d'une simulation, le code calcule plusieurs métriques finales. Elles sont utilisées pour remplir les tableaux et pour comparer les différentes valeurs de λ , de N , de τ et de K .

TABLE 6 – Métriques finales calculées par le simulateur.

Métrique	Calcul et interprétation
Débit final	$n_success/T_{max}$. Il mesure le nombre moyen de paquets transmis avec succès par unité de temps.
Nombre moyen de clients	$total_queue_area/T_{max}$. Il mesure l'occupation moyenne des files d'attente pendant la simulation.
Taux de perte	$n_lost_full/n_arrived$. Il mesure la proportion de paquets refusés à cause des files pleines.
Nombre de collisions	Compteur des stations impliquées dans des collisions. Il mesure l'intensité des conflits d'accès au canal.

4.7 Métriques calculées

Le débit à l'instant t est calculé par

$$\frac{n(t)}{t},$$

où $n(t)$ est le nombre de paquets transmis avec succès jusqu'à l'instant t . La longueur moyenne de file est estimée par intégration temporelle :

$$\bar{L}(T) = \frac{1}{T} \int_0^T \sum_{i=1}^N q_i(t) dt.$$

Le taux de perte vaut

$$p_{\text{perte}} = \frac{\text{paquets refusés car file pleine}}{\text{paquets arrivés}}.$$

5 Validation du simulateur

5.1 Validation de la loi exponentielle

La génération des variables exponentielles est faite par inversion : si U est uniforme sur $]0, 1[$, alors

$$X = -\frac{\ln(U)}{\text{rate}}$$

suit une loi exponentielle de paramètre rate . Une première vérification dans le notebook a donné :

Test de génération exponentielle

Pour $\text{rate} = 2$, la moyenne empirique obtenue sur 10000 échantillons est 0.5028, ce qui est très proche de la moyenne théorique $1/2 = 0.5000$.

5.2 Validation avec un cas sans collision

Le cas $N = 1$ permet de vérifier le simulateur, car une seule station ne peut pas entrer en collision avec une autre. Pour $\lambda = 0.5$, la capacité de transmission vaut 1 paquet par unité de temps, donc le débit attendu est proche de $\min(\lambda, 1) = 0.5$.

TABLE 7 – Test de cohérence pour $N = 1$.

Grandeur	Valeur simulée	Valeur attendue
Débit	0.5171	environ 0.5000
Collisions	0	0
Paquets arrivés	1034	–
Paquets transmis	1034	–

Le résultat confirme que le simulateur respecte le comportement attendu lorsqu'il n'y a aucune collision possible.

5.3 Validation sur une charge modérée

Un deuxième test a été effectué avec $N = 10$ et $\lambda = 0.06$, c'est-à-dire une charge totale $N\lambda = 0.6$. Le simulateur produit alors des collisions et des pertes, ce qui permet de vérifier que les mécanismes de file pleine et de backoff sont bien activés.

TABLE 8 – Test avec charge totale 0.6.

Métrique	Valeur obtenue
Débit mesuré	0.5290
Collisions	1189
Paquets perdus en file pleine	164

6 Résultats expérimentaux

6.1 Evolution temporelle des métriques

La première expérience utilise les paramètres de référence du tableau 2. Le temps de simulation est fixé à $T_{\max} = 3000$ unités de temps. Ce choix permet de dépasser la phase transitoire du début de simulation et d'observer un comportement plus stable. Si la simulation était trop courte, le débit, la taille moyenne des files et le taux de perte dépendraient trop des premières arrivées aléatoires. Avec $T_{\max} = 3000$, les courbes montrent une stabilisation suffisante pour interpréter les métriques finales.

Les paramètres $N = 10$, $K = 10$, $\lambda = 0.06$ et $\tau = 0.5$ ont été choisis pour obtenir une charge totale $N\lambda = 0.6$. Cette charge est inférieure à la capacité théorique du canal, mais elle est assez grande pour créer des collisions et des files d'attente. Elle permet donc d'observer les trois phénomènes importants : succès, collisions et pertes lorsque les files deviennent pleines.

TABLE 9 – Résultats de la simulation temporelle de référence.

Métrique	Valeur
Paquets arrivés	1850
Paquets transmis avec succès	1526
Collisions	1831
Paquets perdus car file pleine	287
Débit estimé $d(N, K, \lambda, \tau)$	0.5087
Nombre moyen de clients	26.6269
Taux de perte	0.1551

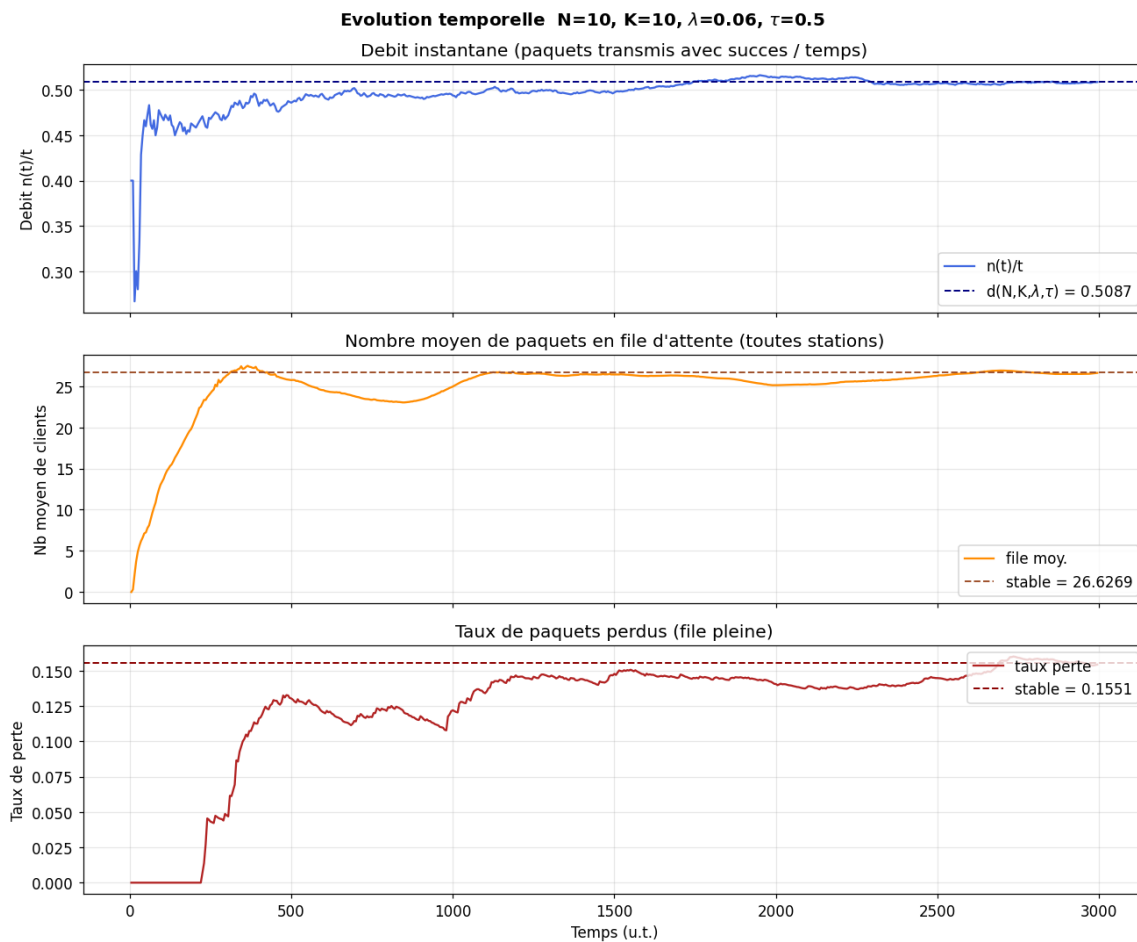


FIGURE 1 – Evolution temporelle du débit, du nombre moyen de clients et du taux de perte pour $N = 10$, $K = 10$, $\lambda = 0.06$, $\tau = 0.5$.

Analyse du premier graphe : débit $n(t)/t$

Le premier sous-graphe représente le débit cumulé $n(t)/t$. Au début, la courbe varie fortement car le nombre de paquets transmis est encore faible. Une seule transmission réussie ou une collision peut donc modifier beaucoup la valeur du rapport $n(t)/t$. Après quelques centaines d'unités de temps, l'effet des premières fluctuations devient moins important, car le nombre total d'événements traités est plus grand.

La courbe se stabilise ensuite autour de 0.51 paquet par unité de temps. Cette stabilisation signifie que le système atteint un régime où, en moyenne, le nombre de transmissions réussies augmente presque linéairement avec le temps. Le débit ne monte pas jusqu'à la capacité maximale théorique 1, car une partie du temps du canal est perdue dans les collisions et dans les périodes d'attente provoquées par le backoff.

Analyse du deuxième graphe : nombre moyen de clients

Le deuxième sous-graphe représente le nombre moyen de paquets présents dans les files d'attente de toutes les stations. Au début, cette valeur augmente rapidement : les stations reçoivent des paquets, mais toutes les demandes ne peuvent pas être servies immédiatement, surtout lorsqu'il y a des collisions. Les files commencent donc à se remplir.

Ensuite, la courbe se stabilise autour de 26.6 paquets. Cela signifie que le système atteint un équilibre entre les arrivées, les transmissions réussies et les pertes dues aux files pleines. Les

files ne deviennent pas vides, mais elles ne continuent pas non plus à croître indéfiniment. Cette stabilisation est due à deux mécanismes : les transmissions réussies retirent des paquets des files, et les files pleines empêchent une croissance illimitée en refusant certains nouveaux paquets.

Analyse du troisième graphe : taux de perte

Le troisième sous-graphe représente le taux de paquets perdus parce que la file de la station concernée est pleine. Au début, le taux est proche de zéro car les files ne sont pas encore remplies. Après une phase transitoire, certaines files atteignent leur capacité maximale $K = 10$, donc des paquets commencent à être refusés.

Le taux de perte converge ensuite vers environ 0.155, c'est-à-dire environ 15.5%. Cette convergence apparaît parce que le système atteint un régime permanent : la fréquence des arrivées, des transmissions réussies, des collisions et des refus devient statistiquement plus régulière. Le taux de perte n'est donc pas constant instantanément, mais sa moyenne cumulée devient stable lorsque le temps de simulation devient suffisamment long.

6.2 Débit en fonction de τ

Cette partie étudie l'effet du paramètre de backoff τ sur le débit, à N , K et λ fixes. Les paramètres utilisés sont :

$$N = 10, \quad K = 10, \quad \lambda = 0.05, \quad T_{\max} = 2000.$$

Chaque point correspond à plusieurs simulations indépendantes, avec un intervalle de confiance à 95%. La figure générée est `plot_debit_vs_tau.png`.

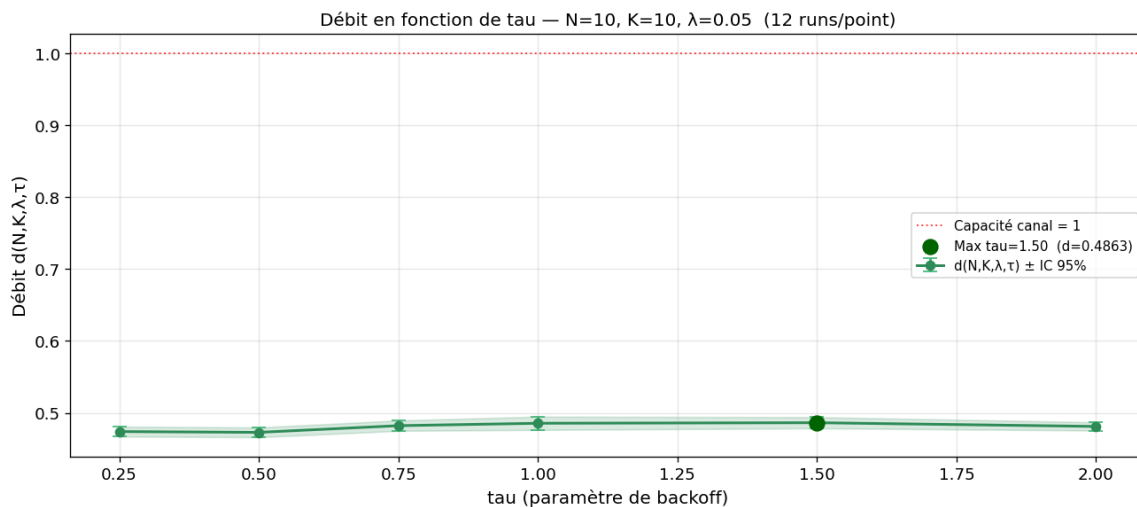


FIGURE 2 – Débit moyen $d(N, K, \lambda, \tau)$ en fonction de τ avec intervalles de confiance à 95%.

Résultat observé

Le meilleur débit moyen observé est obtenu pour :

$$\tau = 1.50, \quad d(10, 10, 0.05, 1.50) = 0.4863, \quad IC_{95\%} = [0.4785, 0.4940].$$

La courbe montre que le débit varie peu avec τ dans cette expérience : il reste proche de 0.47–0.49. Cela signifie que le paramètre τ influence le système, mais que son effet reste modéré pour les valeurs testées.

Pour les petites valeurs de τ , les stations attendent peu après une collision. Elles peuvent donc retenter rapidement, ce qui peut provoquer de nouvelles collisions si plusieurs stations restent en concurrence. Quand τ augmente, les retransmissions sont plus espacées, ce qui peut réduire les collisions et améliorer légèrement le débit.

Cependant, si τ devient trop grand, les stations attendent plus longtemps avant de retenter. Le canal peut alors être moins bien utilisé. C'est pourquoi le meilleur résultat visible est autour de $\tau = 1.5$, puis le débit diminue légèrement pour $\tau = 2.0$. On peut donc interpréter $\tau = 1.5$ comme un bon compromis dans cette configuration.

6.3 Influence de K sur le débit et les pertes

Cette section fait varier la taille de file K par station et observe :

- le débit moyen ;
- le taux de pertes, c'est-à-dire les paquets perdus car une file est pleine.

Les paramètres fixés sont :

$$N = 10, \quad \lambda = 0.06, \quad \tau = 0.5, \quad T_{\max} = 2000.$$

Les valeurs testées sont :

$$K \in \{1, 2, 3, 4, 5, 7, 10, 15, 20, 30\}.$$

Chaque point correspond à plusieurs simulations indépendantes, avec un intervalle de confiance à 95%. La figure générée est `plot_debit_et_pertes_vs_K.png`.

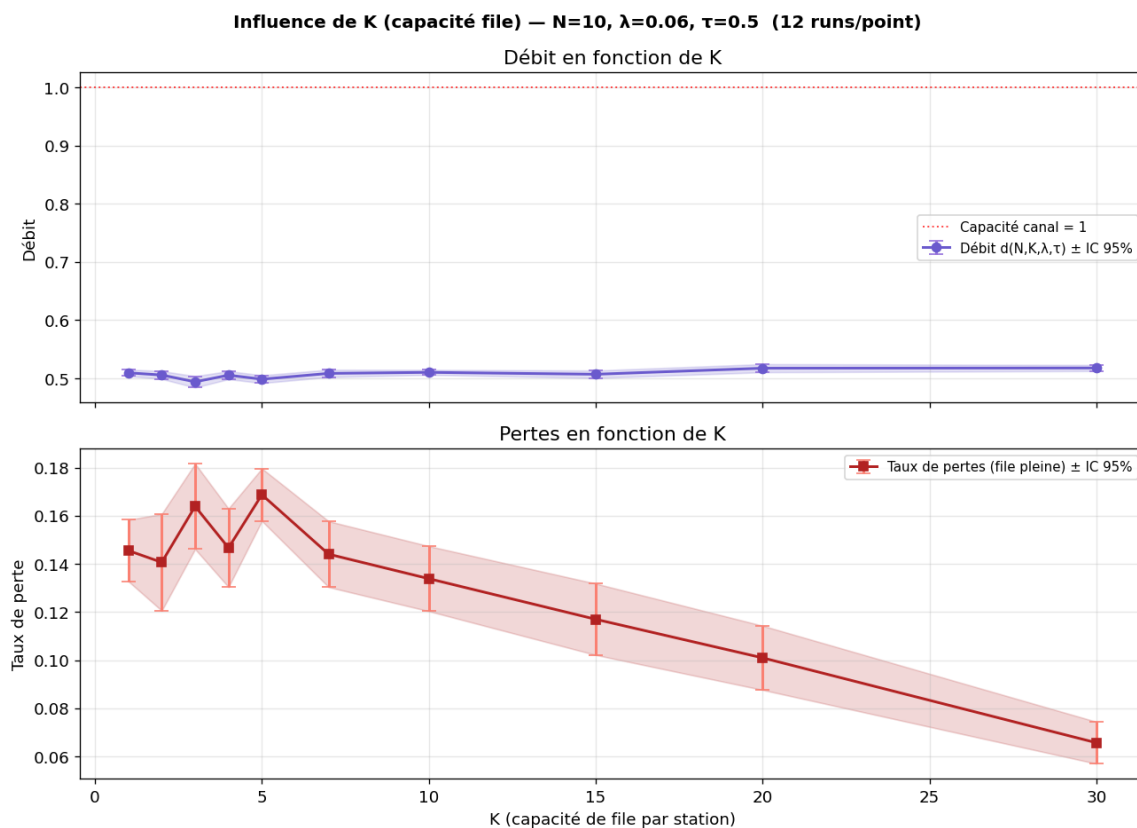


FIGURE 3 – Influence de la capacité de file K sur le débit moyen et sur le taux de pertes.

Résultat observé

Le meilleur résultat visible pour le débit est obtenu pour :

$$K = 30, \quad d(10, 30, 0.06, 0.5) = 0.5175, \quad p_{\text{perte}} = 0.0657.$$

Le premier sous-graphe montre que le débit reste globalement autour de 0.50–0.52. Il ne varie donc pas fortement avec K . Cela s'explique par le fait que la limite principale du débit n'est pas uniquement la taille des files, mais aussi la capacité du canal et les collisions entre stations. Même si les files sont plus grandes, le canal ne peut transmettre qu'un paquet à la fois.

Le deuxième sous-graphe est plus significatif : le taux de perte diminue clairement lorsque K augmente. Avec une petite file, les paquets arrivent rapidement dans des files déjà pleines, donc ils sont refusés. Lorsque K devient plus grand, chaque station peut stocker davantage de paquets en attente, ce qui réduit les pertes.

La conclusion principale est donc que l'augmentation de K améliore surtout la qualité du système en réduisant les pertes. Le débit augmente seulement légèrement, car il reste limité par la capacité du canal et par les collisions.

6.4 Influence de λ sur le débit

Dans cette expérience, les paramètres $N = 10$, $K = 10$, $\tau = 0.5$ sont fixés. On fait varier λ entre 0.01 et 0.13. Chaque point est estimé à partir de 12 répliques indépendantes de durée $T_{\text{max}} = 2000$.

TABLE 10 – Débit moyen en fonction de λ avec IC à 95%.

λ	Charge $N\lambda$	Débit moyen	IC 95%
0.01	0.10	0.0962	[0.0920, 0.1005]
0.02	0.20	0.1959	[0.1885, 0.2033]
0.03	0.30	0.3001	[0.2936, 0.3065]
0.04	0.40	0.3998	[0.3942, 0.4054]
0.05	0.50	0.4754	[0.4641, 0.4867]
0.06	0.60	0.5062	[0.4961, 0.5162]
0.07	0.70	0.5167	[0.5095, 0.5240]
0.08	0.80	0.5172	[0.5102, 0.5242]
0.09	0.90	0.5216	[0.5177, 0.5255]
0.10	1.00	0.5253	[0.5184, 0.5322]
0.11	1.10	0.5296	[0.5262, 0.5330]
0.12	1.20	0.5260	[0.5205, 0.5315]
0.13	1.30	0.5241	[0.5186, 0.5297]

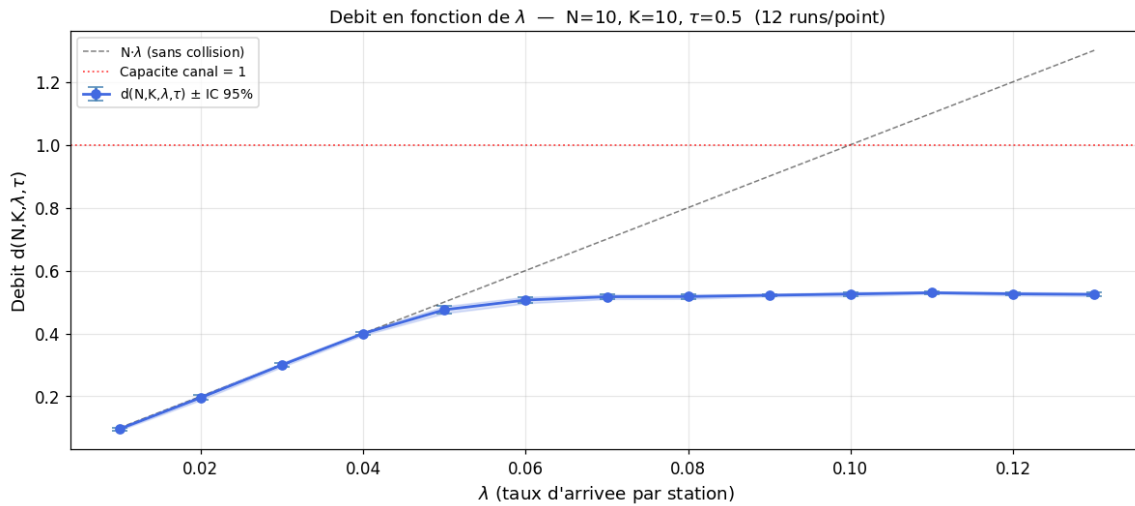


FIGURE 4 – Débit moyen $d(N, K, \lambda, \tau)$ en fonction de λ avec intervalles de confiance à 95%.

Pour les faibles valeurs de λ , le débit suit presque la charge offerte $N\lambda$. Par exemple, lorsque $\lambda = 0.01$, la charge totale vaut 0.10 et le débit obtenu est proche de 0.10. Dans cette zone, le canal est rarement saturé et les collisions sont peu nombreuses ; presque tous les paquets arrivés peuvent être transmis.

Lorsque λ augmente, davantage de stations ont des paquets en attente. Cela améliore d'abord l'utilisation du canal, donc le débit augmente. Cependant, à partir d'une certaine charge, plusieurs stations veulent transmettre en même temps ou presque au même moment. Les collisions deviennent plus fréquentes, ce qui provoque des retransmissions et augmente les délais de backoff. Une partie du temps du canal est alors consommée par des transmissions échouées ou par des attentes avant retransmission.

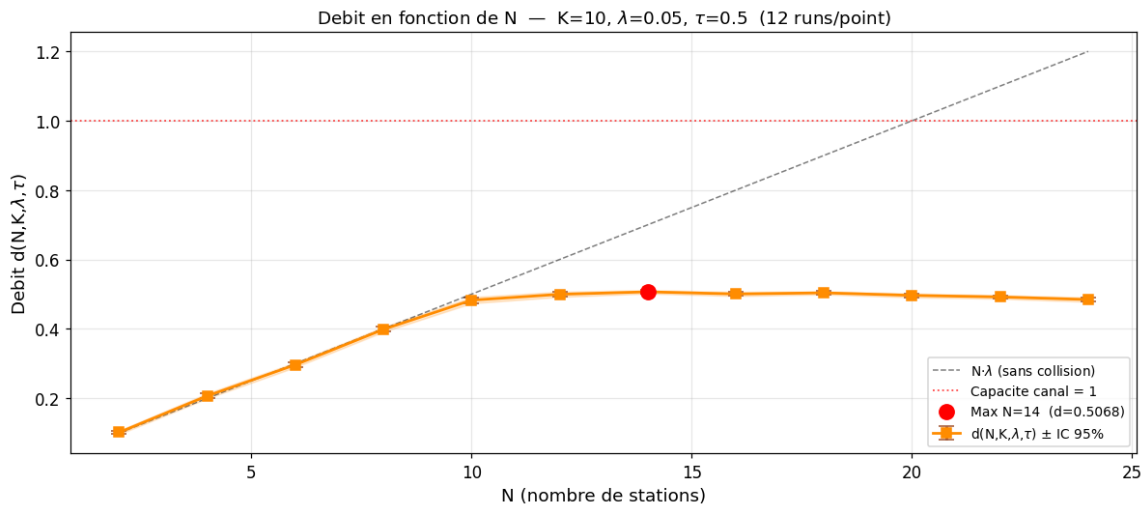
C'est pour cela que le débit se stabilise autour de 0.52–0.53 au lieu de continuer à suivre la droite $N\lambda$. Même si plus de paquets arrivent, le canal ne peut pas transmettre beaucoup plus de paquets avec succès, car les collisions et la saturation des files limitent le gain. L'augmentation de λ ajoute donc surtout de la concurrence et des pertes, pas uniquement du débit utile.

6.5 Influence de N sur le débit

Dans cette expérience, nous fixons $K = 10$, $\lambda = 0.05$, $\tau = 0.5$, puis nous faisons varier N de 2 à 24. Chaque point est calculé avec 12 répliques indépendantes de durée $T_{\max} = 2000$.

TABLE 11 – Débit moyen en fonction de N avec IC à 95%.

N	Charge $N\lambda$	Débit moyen	IC 95%
2	0.10	0.1015	[0.0962, 0.1069]
4	0.20	0.2068	[0.1996, 0.2141]
6	0.30	0.2964	[0.2895, 0.3033]
8	0.40	0.3985	[0.3914, 0.4055]
10	0.50	0.4826	[0.4738, 0.4914]
12	0.60	0.4995	[0.4921, 0.5070]
14	0.70	0.5068	[0.5019, 0.5116]
16	0.80	0.5005	[0.4947, 0.5064]
18	0.90	0.5039	[0.4993, 0.5085]
20	1.00	0.4965	[0.4909, 0.5021]
22	1.10	0.4920	[0.4869, 0.4971]
24	1.20	0.4845	[0.4781, 0.4908]

**FIGURE 5** – Débit moyen $d(N, K, \lambda, \tau)$ en fonction de N avec intervalles de confiance à 95%.

Pour les petites valeurs de N , le canal est sous-utilisé. Il n'y a pas assez de stations pour produire une charge importante, donc le débit est proche de $N\lambda$. Quand N augmente, il y a plus de paquets à transmettre et le canal est mieux exploité. C'est pour cela que le débit augmente fortement entre $N = 2$ et $N = 14$.

Autour de $N = 14$, le système atteint une zone de bon compromis : il y a assez de stations pour utiliser le canal, mais pas encore trop de concurrence. Après cette zone, ajouter des stations n'apporte plus un gain clair. Au contraire, les collisions augmentent, les stations passent plus souvent dans des états de backoff élevés, et certaines files deviennent pleines. Le débit cesse donc d'augmenter et commence à diminuer légèrement.

Pour améliorer le débit lorsque N devient trop grand, plusieurs pistes seraient possibles : augmenter la capacité des files K , ajuster le paramètre de backoff τ , limiter le nombre de stations actives simultanément ou utiliser un mécanisme d'accès au canal plus contrôlé. Dans notre modèle,

le paramètre le plus direct à tester serait τ , car il influence le temps d'attente après collision et donc la probabilité de nouvelles collisions.

7 Détermination du nombre optimal de stations

Pour choisir le meilleur N , nous avons refait une expérience plus précise avec 20 répliques par point et une durée de simulation plus longue, $T_{\max} = 3000$. Les paramètres fixés sont $K = 10$, $\lambda = 0.05$ et $\tau = 0.5$.

TABLE 12 – Détermination de N optimal avec IC à 95%.

N	Débit moyen	IC 95%
2	0.098744	[0.095609, 0.101878]
4	0.201129	[0.197373, 0.204885]
6	0.297254	[0.293945, 0.300563]
8	0.393124	[0.388499, 0.397749]
10	0.474356	[0.469420, 0.479293]
12	0.498912	[0.495478, 0.502346]
14	0.504902	[0.500343, 0.509460]
16	0.503439	[0.499606, 0.507272]
18	0.500184	[0.495573, 0.504795]
20	0.499325	[0.496679, 0.501970]
22	0.494613	[0.492504, 0.496722]
24	0.493859	[0.490283, 0.497435]

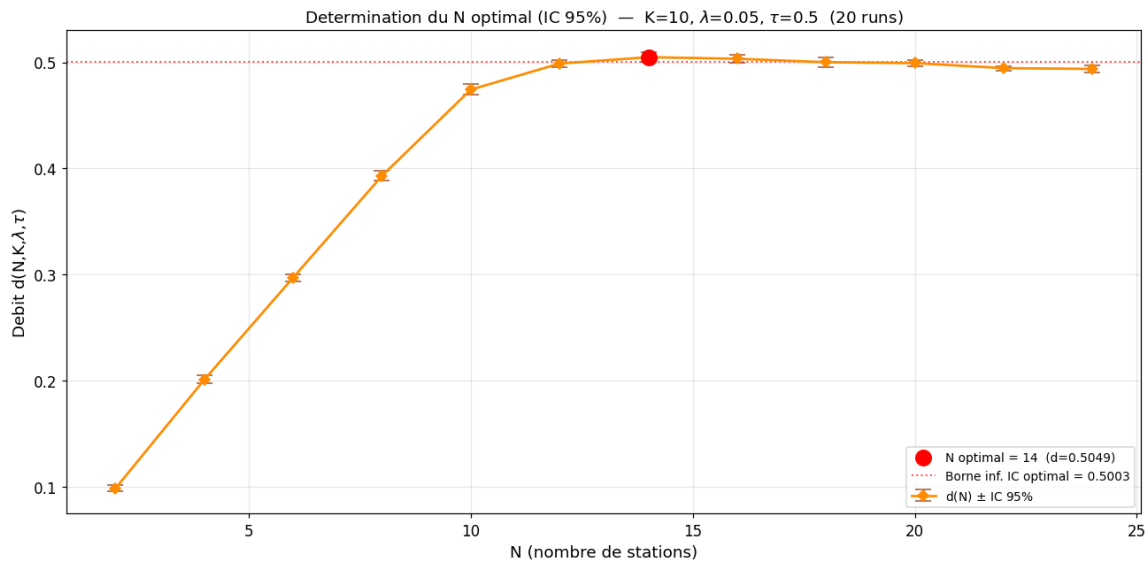


FIGURE 6 – Recherche du nombre de stations maximisant le débit avec intervalles de confiance à 95%.

Résultat principal

Le meilleur débit moyen observé est obtenu pour $N = 14$:

$$d(14, 10, 0.05, 0.5) = 0.504902, \quad IC_{95\%} = [0.500343, 0.509460].$$

La figure montre que le débit augmente fortement pour les petites valeurs de N , puis atteint une zone presque plate autour de $N = 14$, $N = 16$, $N = 18$ et $N = 20$. Le point rouge indique le meilleur débit moyen obtenu, c'est-à-dire $N = 14$. La ligne pointillée horizontale correspond à la borne inférieure de l'intervalle de confiance du meilleur point.

Les valeurs $N = 2, 4, 6, 8, 10, 22, 24$ sont clairement sous-optimales car leur borne supérieure d'intervalle de confiance est inférieure à la borne inférieure de l'intervalle du meilleur point. Les valeurs $N = 16$, $N = 18$ et $N = 20$ restent proches du maximum et leurs intervalles se recouvrent partiellement avec celui de $N = 14$. On peut donc conclure que $N = 14$ est le meilleur choix observé, tout en gardant à l'esprit que les valeurs voisines peuvent donner des performances proches dans cette configuration.

8 Discussion

Les résultats montrent plusieurs phénomènes principaux. D'abord, lorsque la charge est faible, le canal est sous-utilisé et le débit suit presque la charge offerte. Ensuite, lorsque la charge augmente, le débit ne peut pas croître indéfiniment car les collisions et les backoffs consomment du temps. Enfin, avec trop de stations, l'augmentation de la concurrence sur le canal provoque une baisse progressive du débit moyen.

L'étude du paramètre τ montre qu'il existe un compromis entre retransmettre rapidement et attendre suffisamment pour éviter de nouvelles collisions. Dans la configuration testée, le meilleur résultat visible est obtenu pour $\tau = 1.5$, mais les valeurs proches restent assez similaires.

L'étude de la capacité de file K montre que l'augmentation de K réduit surtout les pertes. Le débit augmente seulement légèrement, car la limite principale reste la capacité du canal et les collisions entre stations.

Le choix de $T_{\max} = 3000$ pour l'étude temporelle et pour la recherche de N optimal permet d'observer un régime stabilisé. Les répliques Monte-Carlo, associées aux intervalles de confiance, évitent de baser la conclusion sur une seule trajectoire aléatoire.

La principale limite du modèle est qu'il reste volontairement simple : durée d'émission fixe, stations homogènes, même valeur de λ pour toutes les stations et canal sans bruit autre que les collisions. Ces choix sont suffisants pour étudier le mécanisme de backoff, mais ils pourraient être enrichis dans une version plus réaliste.

9 Conclusion

Nous avons réalisé un simulateur à événements discrets pour un protocole MAC avec *Exponential Backoff*. Le simulateur gère les arrivées de paquets, les files d'attente bornées, les tentatives d'émission, les collisions, les retransmissions et les pertes dues aux files pleines.

Les tests de validation confirment le comportement attendu dans le cas sans collision et dans un cas avec charge modérée. Les expériences montrent que le débit converge vers une valeur stationnaire, que le paramètre λ influence directement la charge du système, et que le nombre de stations N présente un compromis entre utilisation du canal et collisions. Pour les paramètres

$K = 10$, $\lambda = 0.05$ et $\tau = 0.5$, le meilleur nombre de stations observé est $N = 14$, avec un débit moyen de 0.504902 et un intervalle de confiance à 95% de [0.500343, 0.509460].

Les expériences complémentaires montrent aussi que le paramètre τ influence le débit à travers le mécanisme de backoff. Le meilleur débit moyen visible est obtenu pour $\tau = 1.5$, avec $d = 0.4863$. L'étude de K montre que l'augmentation de la capacité de file réduit fortement les pertes : pour $K = 30$, le taux de perte descend à 0.0657 et le débit atteint environ 0.5175.